

NEAREST NEIGHBORS:

A Lazy Learning Algorithm for Classification and Regression

Marco R. Steenbergen

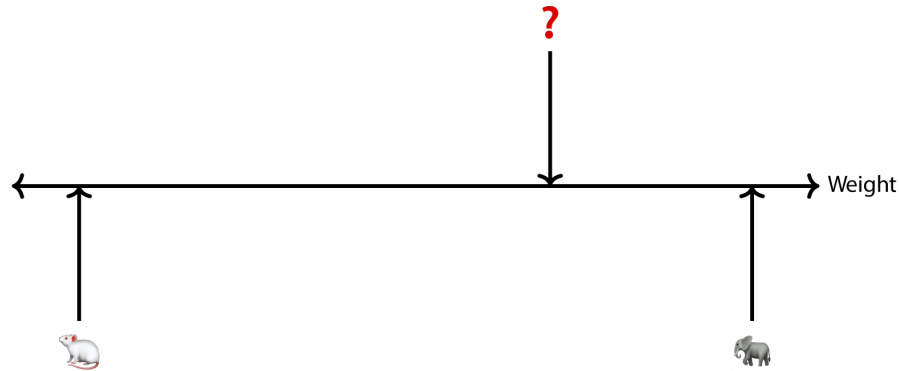
University of Zurich

In this lecture

- The Big Picture
- Distance
- Performance in Classification—Take One
- Nearest Neighbor Classification
- kNN Regression

The Big Picture

Look! An Elephant

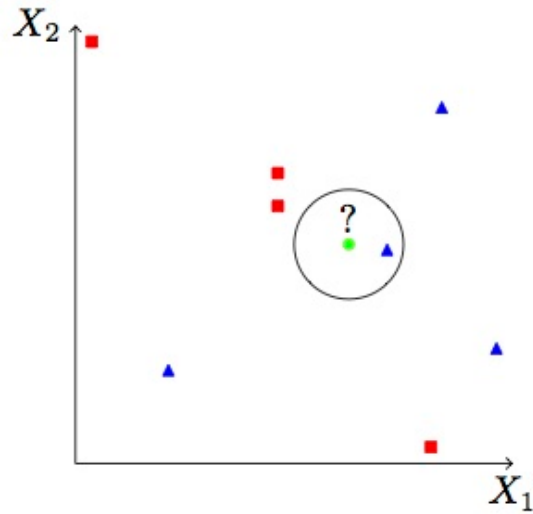


- The sole predictive feature is the animal's weight. There are two **discrete** labels—elephant and mouse → classification task.
- There is a **query**, marked by the question mark. We know the query's weight but not its class.
- We could use the predictive feature and ask: In terms of weight, does the query more resemble the elephant or the mouse?
- The prediction would then be “elephant.”

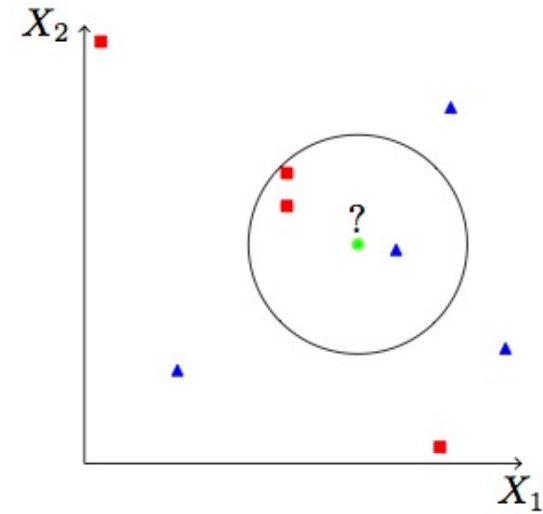
The General Idea of Nearest Neighbors

- Find the training instance that is most similar to the query; this is the **nearest neighbor**.
- Assign the training label to the query.
- More generally, we can find $k = 1, 2, \dots$ nearest neighbors.
- A vote among them determines the prediction.

The k in k -Nearest Neighbors



1-NN: One nearest neighbor is used to make the prediction



3-NN: A vote among the 3 nearest neighbors is taken to make the prediction

Lazy Learning

- The kNN algorithm is a so-called **lazy learner**.
- Lazy learners do not actually generate a model; they simply return a set of labels.
- That makes them quite appealing for such tasks as imputation.
- However, interpretation suffers.

The Road Ahead

- Defining what is $\text{near}(\text{est})$.
- Performance in classification.
- Using the algorithm for classification.
- Using the algorithm for regression.

Distance

Establishing Proximity

- Proximity is the converse of distance.
- Assessing distance in one dimension is simple enough.
- But what do we do when there are many predictive features that all affect distance?
- This is where distance metrics come into play.

Distance Metrics



- Distance metrics map P predictive features onto a single number:
 $\mathbb{R}^P \rightarrow \mathbb{R}^1$.
- Such metrics must meet several requirements:
 1. Reflexivity: If $x = y$ then $d(x, y) = 0$.
 2. Non-negativity: $d(x, y) \geq 0$.
 3. Symmetry: $d(x, y) = d(y, x)$.
 4. Triangle inequality:
 $d(x, z) \leq d(x, y) + d(y, z)$.

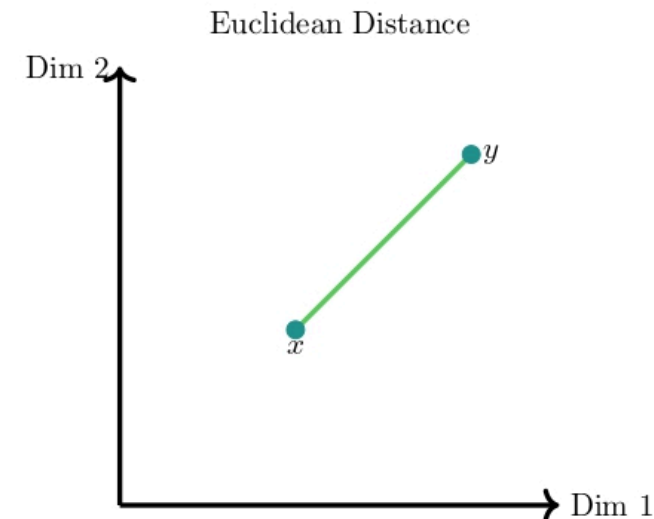
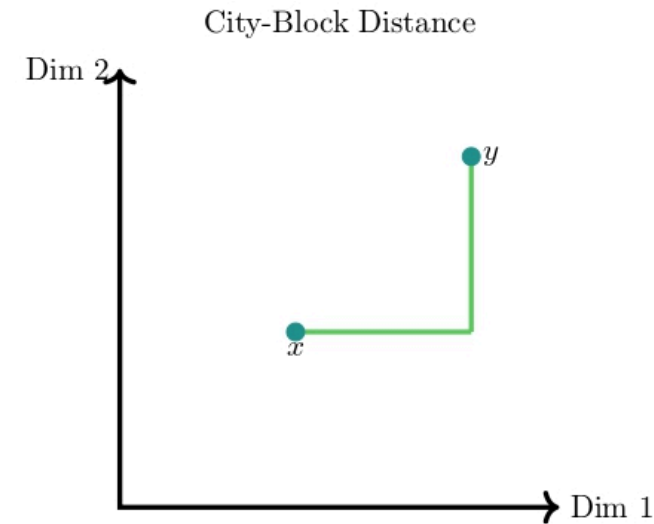
Distance Metrics for Numeric Features

- A commonly used metric is the **Minkowski distance metric**:

$$d(x, y) = \left(\sum_{j=1}^P |x_j - y_j|^q \right)^{\frac{1}{q}}$$

where q is a power set by the researcher.

- Examples include:
 - **City-Block** (Manhattan) distance: $q = 1$.
 - **Euclidean** distance: $q = 2$.
 - **Chebyshev** distance: $q \rightarrow \infty$, which is tantamount to taking the largest distance.



Important

- The Minkowski metric is sensitive to scale differences in predictive features.
- Always ensure that the features are on the same scale, using
 - Normalization
 - Standardization

Distance Metrics for Qualitative Features

- **Hamming distance** counts the number of changes that would need to be made to equalize x and y .
- **Simple matching distance** counts the fraction of disagreements between x and y relative to the number of features.

	NL	SE
Political regime	Democracy	Democracy
Welfare state	Conservative	Social-democratic
Division of power	Unitary State	Unitary State

The hamming distance is 1; only one feature has to be changed so that NL = SE; the simple matching distance is $1/3$.

A Distance Metric for Mixed Features

- Oftentimes, we have both qualitative and quantitative features.
- In this case, we can use the Gower (1971) distance metric. which is the arithmetic mean of
 - Simple matching distances for nominal features.
 - Normalized absolute differences for numeric features.
 - Normalized ranks for ordinal features (Podani, 1999).
- The Gower distance is available through several packages, including [gower](#).

Performance in Classification—Take One

A Simple Classification Problem

- Imagine we divide political regimes into democracies and non-democracies.
- Thus, we have $M = 2$ classes.
- We re-label the classes as
 - Event = democracy
 - Non-event = non-democracy
- Our performance metrics so far have been for numeric scores. How do we assess performance for classifiers?

The Confusion Matrix

Predicted	Observed	
	Event	Non-Event
Event	A	B
Non-Event	C	D

- A = Number of true positives.
- B = Number of false positives.
- C = Number of false negatives.
- D = Number of true negatives.

Different Performance Metrics

- Accuracy = $\frac{A+D}{n}$.
- Sensitivity = $\frac{A}{A+C}$ = Recall.
- Specificity = $\frac{D}{B+D}$.
- Precision = $\frac{A}{A+B}$.
- J = Sensitivity + Specificity – 1 (Youden, 1950).
- Balanced Accuracy = $0.5 \cdot (\text{Sensitivity} + \text{Specificity})$.
- F1 = $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$.

Predicted	Observed	
	Event	Non-Event
Event	A	B
Non-Event	C	D

Accuracy Baselines

- No information rate (NIR): How well can we predict from the marginal label distribution?
- Cohen (1960) κ : How well can we predict by chance?
 - $p_c = \frac{1}{n^2} \cdot [(A + B) \cdot (A + C) + (B + D) \cdot (C + D)]$.
 - $\kappa = \frac{\text{Accuracy} - p_c}{1 - p_c}$.
 - As per Landis & Koch (1977), substantial to perfect agreement requires $\kappa \geq 0.6$.

Nearest Neighbor Classification

Predicting Whether a Country Is a Democracy

Data

Labels

- The 2022 *Economist Intelligence Unit's* democracy index.
- Labels: Democracy versus other.
- Predictive features:
 - Country rank on political stability and the absence of violence.
 - Country rank on government effectiveness.
 - Country rank on the rule of law.

Initial Workflow

Split Sample

Recipe

Model

Flow

```
library(tidymodels)
tidymodels_prefer()
set.seed(10)
rsplit <- initial_split(work_df,
                        prop = 0.75,
                        strata = DEMO)
train <- training(rsplit)
test <- testing(rsplit)
```

Tuning

Re-Sampling

Grid

Metrics

Run

Plot

Details

Optimal

```
set.seed(20)
cv_folds <- vfold_cv(train,
                     v = 10,
                     repeats = 5)
```

Final Workflow

Optimal Fit

Evaluation

Confusion Matrix

Statistics

```
knn_updated <- finalize_model(knn_spec,  
                              knn_opti)  
final_flow <- init_flow %>%  
  update_model(knn_updated)  
final_fit <- final_flow %>%  
  fit(data = train)
```

Exercise

Repeat the analysis, this time not only tuning the number of neighbors but also the Minkowski metric. *Hint:* For the tuning grid, it will be helpful to use the `crossing` function.

kNN Regression

A Simple Algorithm

- Let y be numeric.
- For each query i , we select a set of k neighbors, $\square_i$.
- Our prediction is

$$\psi_i = \frac{1}{k} \sum_{j \in \square_i} y_{j(i)}$$

- Since we average instead of vote, k can be any integer.

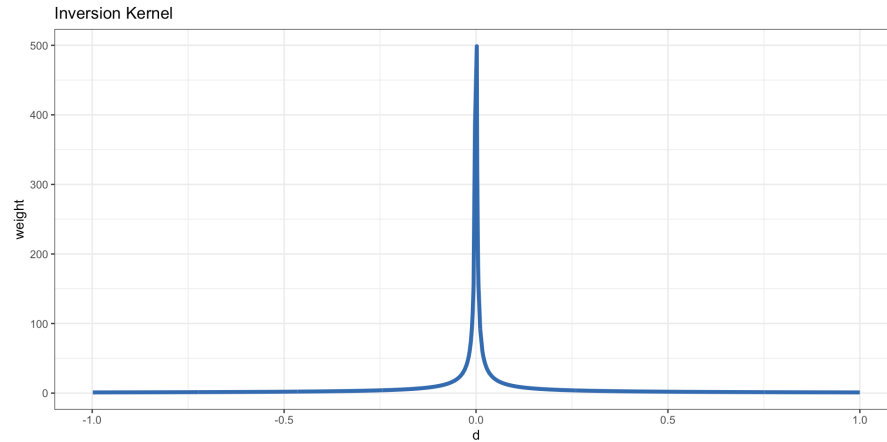
But Wait ... Why Not Weight?

- Why should we give equal weight to all neighbors if some are more alike to the query than others?
- Let us consider all training instances and regulate their importance through a set of weights such that

$$\psi_i = \frac{1}{n_1} \sum_{j=1}^{n_1} q_j y_j \quad (1)$$

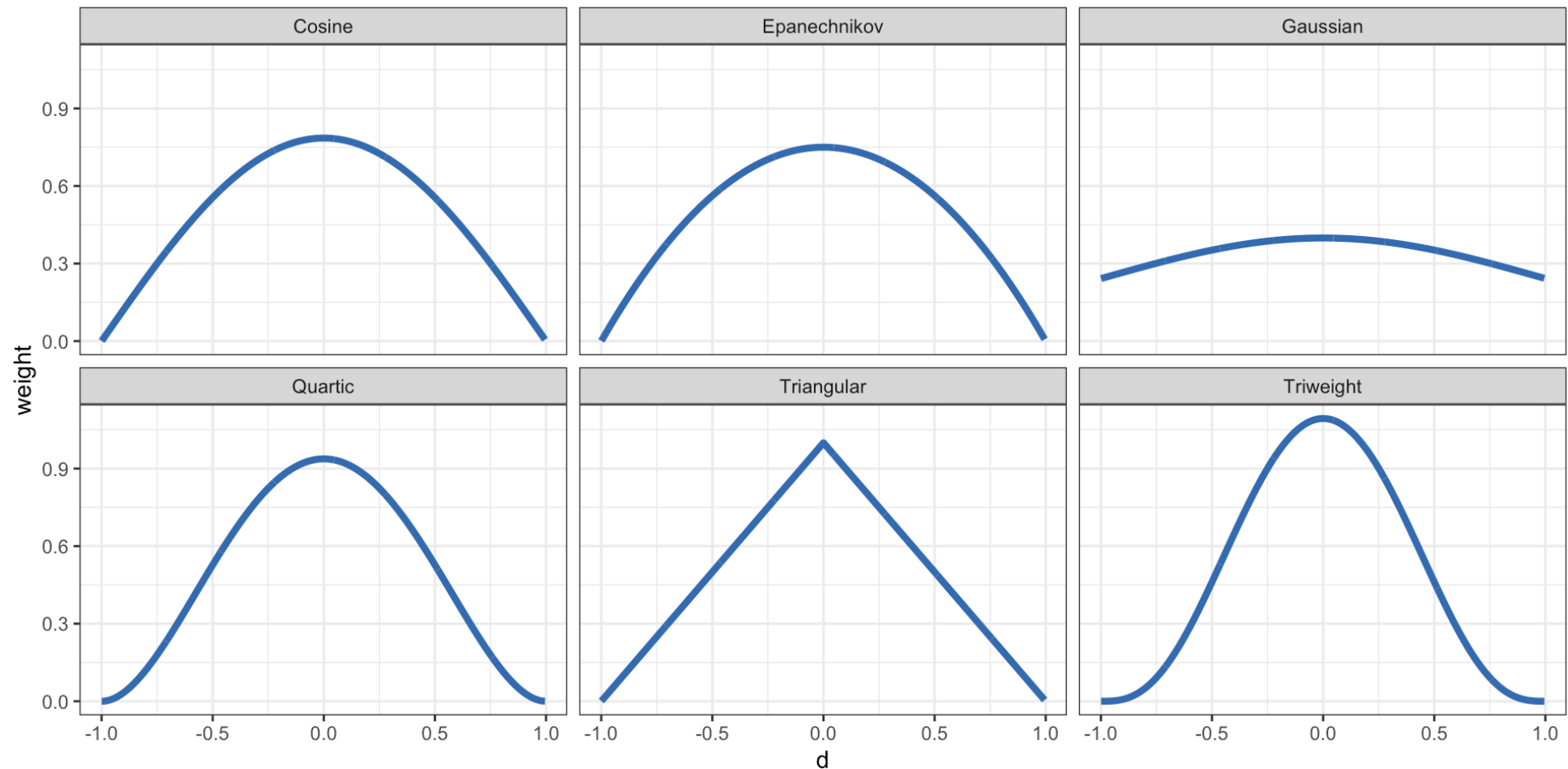
- The weights, q_j , are constructed in such a way that they decrease as a function of $d(i, j)$.
- We can use various kernels for this purpose.

The Principle of Weighting



- The kernel is centered about 0 and gives all of the weight to training units that are perfect copies of the query,
- The weight rapidly decreases as $d(i,j)$ increases.
- This means that the prediction is a function almost exclusively of those training units that are very close to the query.

A Plethora of Kernels



Note: We generally make the kernel a tuning parameter.

And Some More

- The rectangular kernel means that we do not weight at all.
- Samworth (2012) proposes an optimal kernel where the number of neighbors is set to

$$k \cdot \left(\frac{2 \cdot (P + 4)}{P + 2} \right)^{\frac{P}{P+4}}$$

where k is the number of neighbors used under a rectangular kernel.

Setting Up kNN Regression in `tidymodels`

Data

Split

Model

Recipe

Flow

- We use the same predictive features as before.
- However, this time we predict the numeric democracy index.

Tuning the kNN Regression Model

Re-Sampling

Grid

Run

Results

```
set.seed(1891)
cv_folds <- vfold_cv(demo_train, v = 10, repeats = 3)
```

Finalizing the Fit and Making Predictions

Fit

Evaluation

```
knn_updated <- finalize_model(demo_spec,  
                              select_best(knn_tune))  
workflow_new <- demo_flow %>%  
  update_model(knn_updated)  
new_fit <- workflow_new %>%  
  fit(data = demo_train)
```

References

- Cohen, J. (1960). A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement*, 20(1), 37–46. DOI: [10.1177/001316446002000104](https://doi.org/10.1177/001316446002000104)
- Gower, J.C. (1971). A General Coefficient of Similarity and Some of Its Properties. *Biometrics*, 27(4), 857–871. DOI: [10.2307/2528823](https://doi.org/10.2307/2528823)
- Landis, J.R., & Koch, G.G. (1977). The Measurement of Observer Agreement for Categorical Data. *Biometrics*, 33(1), 159–174. DOI: [10.2307/2529310](https://doi.org/10.2307/2529310)
- Podani, J. (1999). Extending Gower's General Coefficient of Similarity to Ordinal Characters. *Taxon*, 48(2), 331–340. DOI: [10.2307/1224438](https://doi.org/10.2307/1224438)
- Samworth, R.J. (2012). Optimal Weighted Nearest Neighbour Classifiers. *The Annals of Statistics*, 40(5), 2733–2763. DOI: [10.1214/12-AOS1049](https://doi.org/10.1214/12-AOS1049)
- Youden, W.J. (1950). Index for Rating Diagnostic Tests. *Cancer*, 3(1), 32–35. DOI: [10.1002/1097-0142\(1950\)3:1<32::aid-cnrcr2820030106>3.0.co;2-3](https://doi.org/10.1002/1097-0142(1950)3:1<32::aid-cnrcr2820030106>3.0.co;2-3)